



Titre Professionnel

Concepteur / Développeur

D'applications

Par CHEVRIER Clément

2023-2024



SOMMAIRE

I. Compétences	3
II. Résumé	4
III. Cahier des charges	5-8
1. Description du besoin.....	5
2. Fonctionnalités.....	8
IV. Spécifications techniques et fonctionnelles	9-18
1. Charte graphique.....	9
2. Maquettage.....	10
3. Conception de la base de données.....	12
4. Mise en place de la base de données.....	15
5. Conventions de nommage.....	17
6. Contraintes techniques.....	18
V. Gestion de projet.....	19-21
1. Planification.....	19
2. Réunions.....	20
3. Versionning.....	21
VI. Réalisations	22-42
1. Scénarios & Visuels.....	22
2. Accès aux données.....	26
3. Application Desktop.....	27
4. Interface Web.....	32
5. Application Mobile.....	37
6. Tests et Sécurité.....	39
7. Déploiement.....	42



I - COMPÉTENCES

N° Fiche AT	Activités types	N° Fiche CP	Compétences professionnelles
1	Concevoir et développer des composants d'interface utilisateur en intégrant les recommandations de sécurité	1	Maquetter une application
		2	Développer une interface utilisateur de type desktop
		3	Développer des composants d'accès aux données
		4	Développer la partie front-end d'une interface utilisateur web
		5	Développer la partie back-end d'une interface utilisateur web
2	Concevoir et développer la persistance des données en intégrant les recommandations de sécurité	6	Concevoir une base de données
		7	Mettre en place une base de données
		8	Développer des composants dans le langage d'une base de données
3	Concevoir et développer une application multicouche répartie en intégrant les recommandations de sécurité	9	Collaborer à la gestion d'un projet informatique et à l'organisation de l'environnement de développement
		10	Concevoir une application
		11	Développer des composants métier
		12	Construire une application organisée en couches
		13	Développer une application mobile
		14	Préparer et exécuter les plans de tests d'une application
		15	Préparer et exécuter le déploiement d'une application



II - RÉSUMÉ

To validate the skills required to obtain the title of application designer and developer, we conceptualized and developed an interactive questionnaire, based on the principle of an innovative and engaging serious game, for the EvoFlux company.

The main objective is to create an interactive and immersive experience, offering potential future clients a fun opportunity to carry out a quick diagnosis of their company's various management processes, and identify possible areas for improvement, so that EvoFlux can present them with solutions tailored to their needs.

Partly in groups and partly individually, this project is the result of learning during the training course.

First, we modeled and mocked up the application based on the customer's request.

We then developed the desktop application in Python, as well as with various libraries such as TkInter, VLC or Pillow.

To do this, I had to create a database using MySQL instantiated on the company's server.

Based on this Desktop application, we reproduced the application in web format using Flask with HTML/CSS and JS.

Then, we created a mobile application in React Native, adding functionality so that EvoFlux could also be easily accessed on the phone.



III – CAHIER DES CHARGES

1. Description du besoin :

Objectif du Projet :

Le client, EvoFlux, a exprimé le besoin de créer un outil de diagnostic interactif visant à évaluer et améliorer les compétences en gestion d'entreprise de ses utilisateurs. Ce projet se distingue par sa nécessité de fonctionner sur plusieurs supports, incluant une application Desktop, une application Mobile, ainsi qu'une intégration directe sur le site web existant de l'entreprise. Cette approche multiplateforme permet d'assurer une accessibilité optimale, permettant aux utilisateurs d'interagir avec l'outil de manière fluide et continue, qu'ils soient au bureau, en déplacement, ou en ligne.

Un aspect crucial du projet est le respect des normes du Règlement Général sur la Protection des Données (RGPD). L'outil devra intégrer des fonctionnalités pour garantir la protection des données personnelles des utilisateurs. Il devra donc collecter uniquement les informations nécessaires à l'évaluation, tout en assurant la transparence sur la manière dont ces données seront utilisées. Cette collecte et traitement des données visent à fournir à EvoFlux une pré-évaluation des pratiques de gestion des entreprises prospectives, tout en respectant les droits et la vie privée des utilisateurs.

EvoFlux se positionne comme un acteur clé dans le domaine des logiciels de gestion, adaptées aux besoins spécifiques des entreprises. Contrairement à une approche axée sur le volume, l'entreprise privilégie une relation proactive avec ses clients. EvoFlux travaille en étroite collaboration avec eux pour développer des solutions personnalisées, afin d'améliorer leur productivité, optimiser leur gestion, et répondre aux contraintes réglementaires, comme l'automatisation des processus de facturation.

Ce projet d'outil de diagnostic interactif s'inscrit parfaitement dans la stratégie d'EvoFlux. Il vise non seulement à fournir une évaluation détaillée des compétences en gestion, mais aussi à renforcer l'accompagnement des entreprises dans leur développement. En proposant un outil accessible sur plusieurs plateformes, conforme aux réglementations en vigueur, et capable de fournir des insights précieux sur les pratiques de gestion, EvoFlux renforce son engagement à soutenir ses clients dans l'amélioration continue de leur gestion d'entreprise.



III – CAHIER DES CHARGES

Les utilisateurs du projet :

Les destinataires du projet se divisent en deux groupes principaux : les **administrateurs** et les **clients**.

❖ Les administrateurs (décisionnaires, membres d'EvoFlux) :

Ils jouent un rôle central dans la gestion et l'évolution de l'outil. Bien qu'ils ne possèdent pas nécessairement des compétences en programmation, ils disposent d'une vue d'ensemble complète des fonctionnalités de l'outil et ont accès à toutes ses capacités. Ils bénéficient d'une documentation détaillée qui leur permet de modifier et d'adapter l'outil selon les besoins futurs. Par exemple, ils peuvent ajouter de nouvelles questions ou scénarios pour enrichir l'évaluation proposée par l'outil, assurant ainsi que celui-ci reste pertinent et adapté aux évolutions des exigences et des besoins des utilisateurs.

❖ Les clients :

Les clients, quant à eux, sont les utilisateurs finaux de l'outil. Ils peuvent ne pas être particulièrement à l'aise avec les technologies informatiques, ce qui peut les rendre susceptibles de commettre des erreurs involontaires lors de l'utilisation de l'outil. Il est donc crucial de les guider de manière claire et intuitive tout au long de leur interaction avec l'outil. Des messages d'avertissement et des instructions claires doivent être intégrés pour les informer des conséquences potentielles de leurs actions, afin de minimiser les erreurs et améliorer l'expérience utilisateur.

De plus, il est important de prendre en compte que les clients peuvent présenter des comportements malveillants, ce qui nécessite une vigilance accrue lors de la réception de données de leur part. Des mécanismes de sécurité robustes doivent être en place pour protéger l'intégrité des données et éviter les abus. Enfin, il est également essentiel de concevoir l'outil de manière à s'adapter aux éventuelles déficiences des utilisateurs, en intégrant des fonctionnalités d'accessibilité et en assurant que l'outil reste fonctionnel et utile pour un large éventail de personnes, quelles que soient leurs compétences techniques ou leurs besoins spécifiques.



III – CAHIER DES CHARGES

Validation par le client :

La validation par le client est un aspect essentiel du processus de développement. Chaque document et chaque outil produit lors de la conception de l'application doivent être soumis à l'approbation du client de manière individuelle. Cette procédure garantit que tous les éléments du projet répondent aux attentes et aux exigences définies.

Les développeurs ont la liberté d'utiliser toutes les technologies et langages nécessaires pour atteindre les objectifs du projet, à condition que chaque choix technologique soit bien documenté. Cette documentation permet au client de comprendre les décisions techniques prises et assure la transparence tout au long du développement.

Avant le début du développement proprement dit, la maquette de l'application doit être validée par le client. Cette étape est cruciale pour s'assurer que la conception répond aux attentes et que toutes les fonctionnalités prévues sont bien représentées. La validation de la maquette constitue un point de référence pour l'ensemble du projet et permet d'identifier et de corriger les éventuelles divergences avant de commencer la phase de développement.

Les documents finaux doivent strictement correspondre au projet développé. Toute modification apportée au projet doit être communiquée au client dès que possible. Il est important que le client soit informé des raisons de ces modifications afin de pouvoir les évaluer et donner son approbation ou ses instructions pour d'éventuels ajustements. Cette communication rapide et transparente est essentielle pour maintenir la confiance et garantir que le produit final est conforme aux attentes du client.



III – CAHIER DES CHARGES

2. Fonctionnalités attendues :

Les fonctionnalités attendues pour l'outil de diagnostic interactif incluent plusieurs éléments clés destinés à offrir une évaluation complète et personnalisée de la gestion d'entreprise :

- ❖ Questionnaire interactif : L'outil doit proposer un questionnaire interactif conçu pour évaluer en profondeur le contexte et les pratiques de gestion actuelles de l'entreprise de l'utilisateur. Ce questionnaire est structuré de manière à recueillir des informations détaillées sur les processus de gestion, les pratiques en cours, et les défis rencontrés. Il doit être convivial et facile à naviguer pour permettre aux utilisateurs de fournir des réponses précises sans difficulté.
- ❖ Proposition de scénarios et de situations : En fonction des réponses fournies, l'outil doit générer des scénarios et des situations pertinents pour aider l'utilisateur à mieux comprendre les implications de ses pratiques de gestion. Ces scénarios doivent être conçus pour illustrer des situations concrètes auxquelles l'utilisateur pourrait être confronté, offrant ainsi une expérience enrichissante et adaptée aux besoins spécifiques de chaque utilisateur.
- ❖ Score personnalisé : L'outil doit fournir un score personnalisé basé sur les réponses du questionnaire. Ce score doit être détaillé et divisé en différents thèmes ou catégories, permettant ainsi une évaluation approfondie des diverses dimensions de la gestion d'entreprise. Ce score vise à identifier les points forts et les axes d'amélioration pour aider l'utilisateur à optimiser sa gestion.
- ❖ Coordonnées de contact : Les utilisateurs doivent avoir la possibilité de laisser leurs coordonnées à la fin du questionnaire. Cette fonctionnalité permet à EvoFlux de recontacter les utilisateurs pour offrir un accompagnement supplémentaire, répondre à d'éventuelles questions, ou discuter des résultats du diagnostic. Cette option doit être simple à utiliser et garantir la confidentialité des informations fournies.



IV – SPÉCIFICATIONS TECHNIQUES ET FONCTIONNELLES

1. Charte graphique :

La charte graphique des différentes applications devra strictement suivre les directives visuelles d'EvoFlux, qui sont fondées sur les couleurs du logo de l'entreprise. Cette cohérence visuelle assurera que toutes les applications reflètent l'identité de marque d'EvoFlux de manière homogène et reconnaissable.

		
# 9DCDD1	# 244C7B	# 57A8B0
		
# 303170	# E2DEDF	# 176585

Les couleurs utilisées dans les applications doivent être choisies en respectant la palette définie par le logo de l'entreprise. Cette palette doit être appliquée de manière cohérente à travers toutes les interfaces pour garantir une expérience utilisateur harmonieuse et une reconnaissance visuelle immédiate.

En plus de respecter la charte graphique en termes de couleurs, il est essentiel de veiller à ce que la police d'écriture soit non seulement conforme aux lignes directrices de la marque, mais également optimisée pour l'accessibilité. La lisibilité doit être une priorité, notamment pour les personnes mal voyantes. Cela implique de choisir des polices claires et suffisamment grandes, avec un contraste adéquat entre le texte et le fond. Des ajustements doivent être effectués pour garantir que le texte reste lisible dans diverses conditions d'éclairage et pour les utilisateurs ayant des déficiences visuelles.



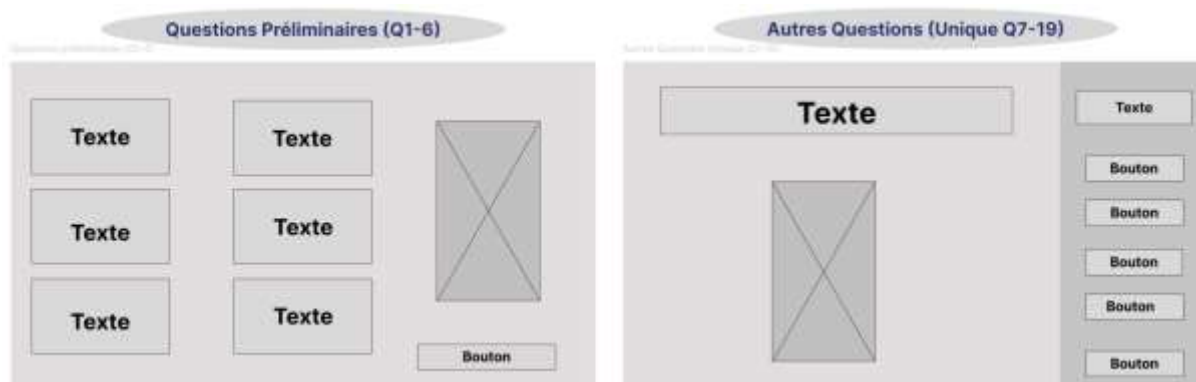
IV – SPÉCIFICATIONS TECHNIQUES ET FONCTIONNELLES

2. Maquettage :

Le maquettage de l'application est réalisé à l'aide de l'outil Figma, ce qui permet de présenter les premiers visuels de l'application finale à EvoFlux. Cette phase de maquettage est cruciale pour donner au client une vue d'ensemble de l'apparence et de la disposition de l'application avant le début du développement proprement dit.

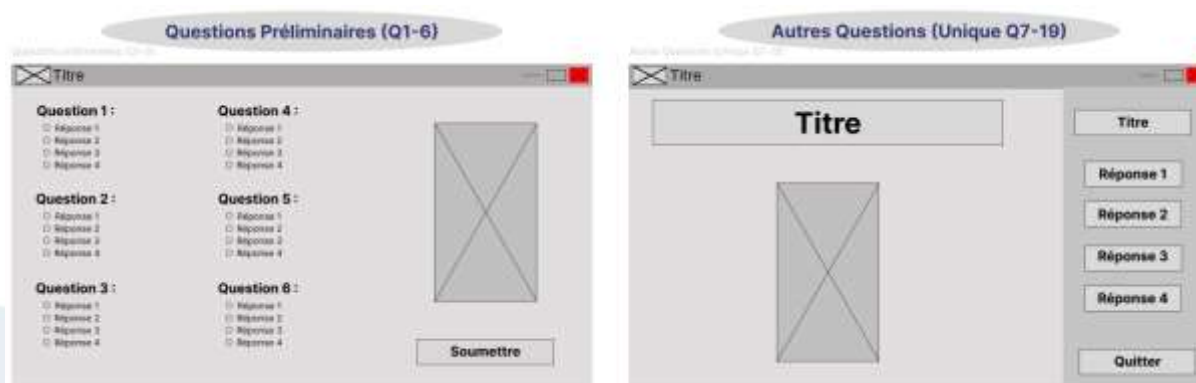
L'étape finale du maquettage doit se rapprocher autant que possible de l'application qui sera effectivement développée. Cela signifie que les maquettes fournies à EvoFlux refléteront fidèlement le design, la disposition, et les fonctionnalités prévues pour l'application. Ce niveau de détail permet de valider les choix de conception et d'apporter les ajustements nécessaires avant le début du développement.

En collaboration avec le client, nous avons donc commencé par réaliser le zoning des visuels souhaités, afin d'avoir une première idée de ce que à quoi ressemblera l'application finale.



Une fois ceci fait, nous sommes passés à la conception du wireframe. Cette étape consiste à améliorer ce qui a été fait lors du zoning, pour se rapprocher encore plus de la maquette et permet de voir plus clairement les différents éléments de chaque visuel.

Voici le wireframe des deux visuels montrés précédent pour le zoning :





IV – SPÉCIFICATIONS TECHNIQUES ET FONCTIONNELLES

Après la validation des visuels de zoning par EvoFlux, nous avons pu passer à la réalisation de la maquette détaillée. Cette maquette fournit une représentation visuelle complète de l'application telle qu'elle sera une fois terminée. Elle intègre toutes les fonctionnalités et éléments de design validés, offrant une vision précise et fidèle du produit final.

Cette étape est essentielle pour s'assurer que le design répond pleinement aux attentes du client avant de commencer le développement. La validation de cette maquette permet de confirmer que tous les aspects du design sont conformes aux spécifications et aux exigences d'EvoFlux, garantissant ainsi une application cohérente et alignée avec la vision du client.



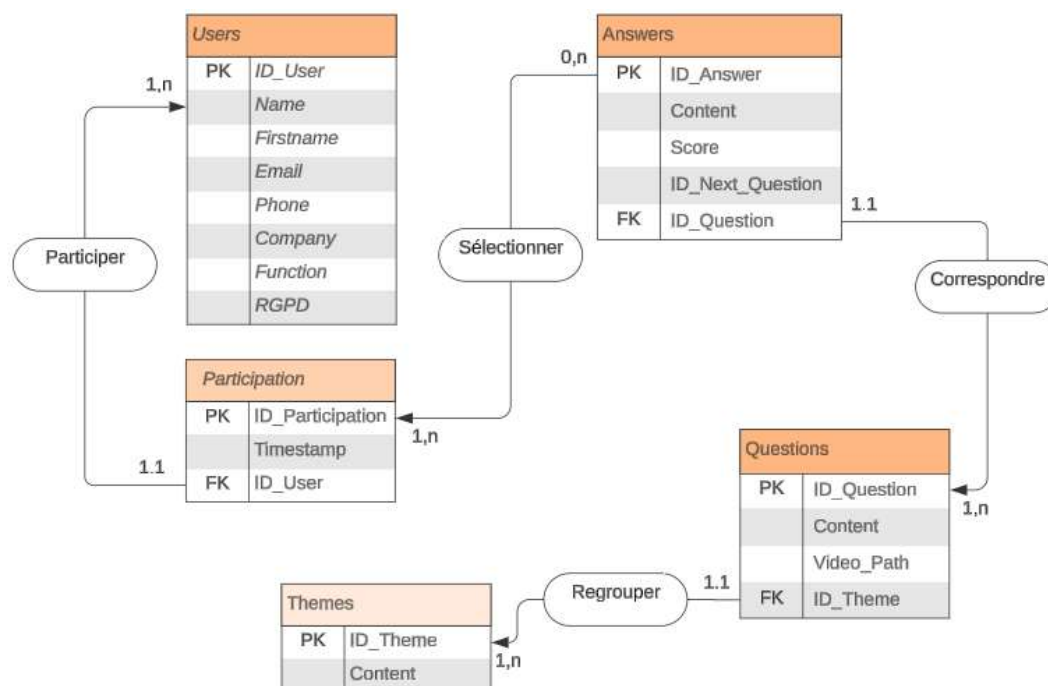


IV – SPÉCIFICATIONS TECHNIQUES ET FONCTIONNELLES

3. Conception de la base de données :

Pour la modélisation de la base de données, nous avons choisi d'utiliser Lucidchart, un outil qui facilite la création et la gestion des différents aspects de la modélisation selon le modèle MERISE. Ce modèle a été privilégié en raison de sa capacité à fournir une structure claire et hiérarchisée, essentielle pour la conception d'une base de données robuste et évolutive. MERISE se distingue par sa flexibilité, permettant d'adapter le système aux besoins changeants sans nécessiter une refonte complète. La structure ainsi créée servira de socle solide pour notre application.

Le processus de modélisation a débuté par la création du Modèle Conceptuel de Données (MCD). Ce modèle permet de représenter les besoins du système de manière abstraite, en identifiant les entités et les relations (entités-associations) sans entrer dans les détails techniques. Le MCD nous a permis de visualiser les éléments clés du système et leurs interactions de manière simplifiée, facilitant ainsi la compréhension et la validation des besoins fonctionnels.





IV – SPÉCIFICATIONS TECHNIQUES ET FONCTIONNELLES

Une fois le MCD établi, nous avons procédé à la modélisation du Modèle Logique de Données (MLD). Cette étape consiste à transformer le MCD en un modèle plus détaillé et technique, en spécifiant les tables de la base de données, leurs attributs, ainsi que les clés primaires et étrangères associées. Le MLD respecte les contraintes du modèle relationnel, assurant ainsi la cohérence et l'intégrité des données. Cette phase est cruciale pour définir la structure exacte de la base de données et préparer sa mise en œuvre effective dans un système de gestion de base de données (SGBD).

En suivant ces étapes, nous avons pu créer une base de données bien structurée, capable de répondre aux exigences fonctionnelles du projet tout en facilitant son évolutivité et sa maintenance future.

Users: ID_User, Name, Firstname, Email, Phone, Company, Function, RGPD

Participation: ID_Participation, Timestamp, ID_User

Participation Answers: ID_Answer, ID_Participation

Answers: ID_Answer, Content, Score, ID_Next_Question, ID_Question

Questions: ID_Question, Content, Video_Path, ID_Theme

Themes: ID_Theme, Content

La dernière étape de notre conception de base de données consiste à traduire le Modèle Logique de Données (MLD) en un Modèle Physique de Données (MPD). Ce modèle représente graphiquement la structure de la base de données telle qu'elle sera implémentée sur le Système de Gestion de Base de Données (SGBD) sélectionné. Le MPD permet de concrétiser le design en un schéma qui reflète les tables, les clés et les relations qui seront réellement utilisées dans la base de données.



IV – SPÉCIFICATIONS TECHNIQUES ET FONCTIONNELLES

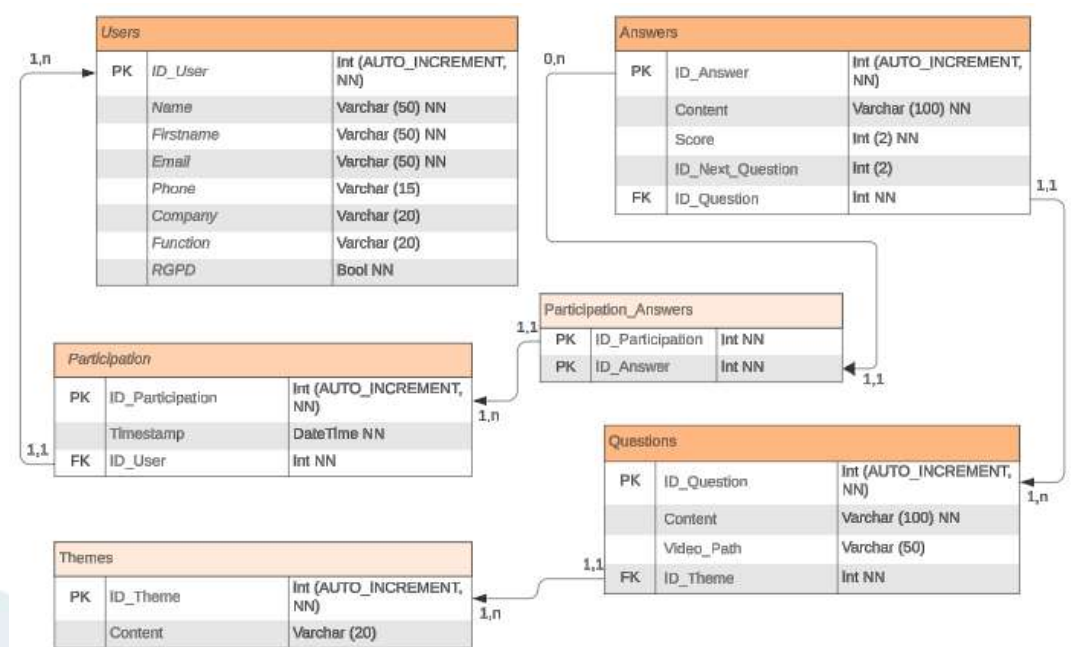
Pour illustrer le schéma physique, prenons l'exemple de la table « Users », qui est utilisée pour stocker les informations des utilisateurs. Les données collectées seront saisies par l'utilisateur à la fin du diagnostic, avec une option de saisie anonyme si l'utilisateur le préfère.

À partir de la table « Users », une relation est établie vers la table « Participation ». Cette table attribue un identifiant unique à chaque utilisateur et enregistre la date et l'heure de leur participation. Cette relation permet de suivre les interactions des utilisateurs avec le système.

Les réponses des utilisateurs sont ensuite enregistrées dans la table « Answers ». Chaque réponse est associée à une participation spécifique via la table « Participation_Answers ». Cette table fait le lien entre les réponses et les participations, facilitant ainsi la gestion des réponses au diagnostic.

Chaque réponse est liée à une question spécifique, stockée dans la table « Questions ». Les questions sont organisées en différents thèmes, regroupés dans la table « Themes ». Cette hiérarchisation permet de structurer les questions de manière cohérente et d'améliorer l'organisation des données au sein du diagnostic.

Ainsi, le MPD fournit une vue détaillée et opérationnelle de la base de données, facilitant son implémentation et assurant que toutes les relations et contraintes définies dans le MLD sont respectées dans le SGBD. Cette étape est essentielle pour garantir l'intégrité et l'efficacité du stockage des données dans le système.

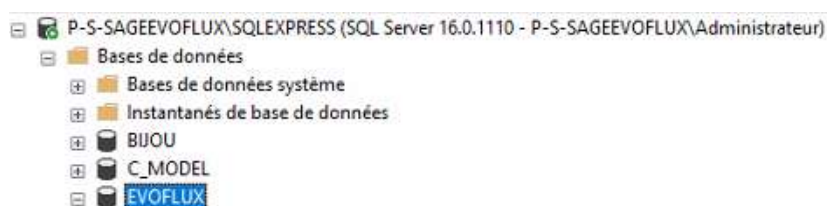




IV – SPÉCIFICATIONS TECHNIQUES ET FONCTIONNELLES

4. Mise en place de la base de données :

La mise en place de la base de données a été réalisée sur le serveur de production de l'entreprise, situé dans une zone démilitarisée (DMZ) pour garantir la sécurité et l'intégrité des données. Le système de gestion de base de données (SGBD) choisi pour ce projet est SQL Server Management Studio (SSMS), conformément aux habitudes de l'entreprise qui utilise régulièrement cet outil.



Pour faciliter le déploiement de la base de données, un script de création a été développé. Ce script permet de configurer rapidement et efficacement la base de données sur le serveur.

En parallèle, un programme en Python a été mis en place pour insérer les données nécessaires dans la base. Ces données, stockées sous forme de fichiers .csv, incluent des informations essentielles telles que les questions, les réponses, et les thèmes. L'utilisation des fichiers .csv simplifie la maintenance et l'intégration des données, tout en assurant un bon fonctionnement du projet.

```
CREATE TABLE Users (
    ID_User INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
    Name VARCHAR(50) NOT NULL,
    Firstname VARCHAR(50) NOT NULL,
    Email VARCHAR(100) NOT NULL,
    Phone VARCHAR(15),
    Company VARCHAR(20),
    Function VARCHAR(20),
    Conditions BOOL NOT NULL DEFAULT 0,
    RGPD BOOL NOT NULL DEFAULT 0
);

-- Create Themes table
CREATE TABLE Themes (
    ID_Theme INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
    Content VARCHAR(20) NOT NULL
);
```

```
# Insertion des données depuis le fichier CSV pour la table Themes
with open('data/themes.csv', 'r', newline='', encoding='utf-8') as file:
    csv_reader = csv.DictReader(file)
    for row in csv_reader:
        conn.execute("INSERT INTO Themes (ID_Theme, Content) VALUES (?, ?)",
                    (row['ID_Theme'], row['Content']))
```



IV – SPÉCIFICATIONS TECHNIQUES ET FONCTIONNELLES

Pour garantir l'intégrité et la sécurité des données, un script batch a été déployé en tant que routine sur le serveur. Ce script effectue une sauvegarde quotidienne de la base de données, et des fichiers de logs sont générés pour suivre ces sauvegardes. Cette procédure assure la protection des données contre la perte ou la corruption.



De plus, des tests unitaires ont été créés pour vérifier l'insertion et la modification des données dans la base de données. Ces tests permettent de s'assurer que les opérations de manipulation des données se déroulent correctement et que la base de données répond aux exigences fonctionnelles et de performance du projet.

```
def test_conditions_and_rgpd_defaults(self):
    # Verify that the default values for Conditions and RGPD are correct
    self.cur.execute("INSERT INTO Users (Name, Firstname, Email) VALUES (?, ?, ?)", ('Smith', 'Alice', 'alice@example.com'))
    self.conn.commit()
    self.cur.execute("SELECT RGPD FROM Users WHERE Name='Smith' AND Firstname='Alice'")
    user = self.cur.fetchone()
    self.assertIsNotNone(user)
    self.assertEqual(user[0], 0) # Check that RGPD is 0 by default
```




IV – SPÉCIFICATIONS TECHNIQUES ET FONCTIONNELLES

5. Conventions de nommage :

La convention de nommage utilisé dans le code sera le Snake Case, convention de nommage qui consiste à séparer les mots par des underscores (_) et de mettre toutes les lettres en minuscules.

```
buttons = []
y_position = 0.17
button_width = 25
button_height = 3
```

Cependant, concernant la partie base de données, le Pascal Snake Case, convention de nommage qui consiste à relier tous les mots avec des underscores (_) et de mettre la première lettre de chaque mot en majuscule, sera privilégié.

Nous veillerons à éviter l'utilisation de mots réservés, de caractères spéciaux, et d'abréviations non standard dans les noms des variables et des objets. Cette pratique contribue à réduire les risques de conflits et d'ambiguïtés dans le code et la base de données.

De plus, tout le code sera rédigé et commenté en anglais. Cette uniformité linguistique assure que le code soit compréhensible par une audience internationale et facilite la collaboration et la maintenance, en particulier dans un environnement de développement global.

```
40  -- Create Questions table
41  CREATE TABLE Questions (
42      ID_Question INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
43      Content VARCHAR(100) NOT NULL,
44      Video_Path VARCHAR(50),
45      ID_Theme INTEGER NOT NULL,
46      FOREIGN KEY (ID_Theme) REFERENCES Themes(ID_Theme)
47  );
```



IV – SPÉCIFICATIONS TECHNIQUES ET FONCTIONNELLES

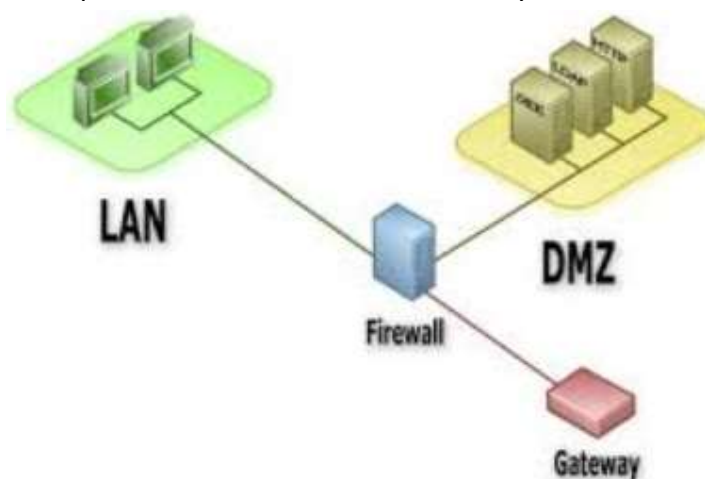
6. Contraintes techniques :

Pour assurer la réalisation complète du projet, nous avons sélectionné les outils les plus appropriés pour son développement, sa maintenance et sa mise en production. Le projet dans son ensemble ainsi que sa base de données seront initialement stockés sur un serveur local au sein de l'entreprise.

La base de données sera intégrée dans une Zone Démilitarisée (DMZ), une configuration essentielle pour garantir le respect des critères de sécurité définis par l'analyse de risque DICP (Déclaration d'Intérêt et de Conformité à la Protection). Cette approche de sécurisation est conçue pour protéger les données non seulement des menaces extérieures, mais aussi des éventuels risques au sein de l'environnement local de l'entreprise.

En plaçant la base de données dans une DMZ, nous assurons une isolation rigoureuse des systèmes critiques, réduisant ainsi les vulnérabilités potentielles aux attaques externes et améliorant la résilience du système. Les mesures de sécurité mises en place comprennent des pare-feux, des systèmes de détection d'intrusion, et des contrôles d'accès renforcés, garantissant une protection optimale des données tout au long de leur cycle de vie.

Cette architecture contribue non seulement à la conformité avec les exigences de sécurité, mais également à la performance et à la fiabilité du système dans son ensemble.





V – GESTION DE PROJET

1. Planification

Pour la réalisation de ce projet, j'ai eu l'occasion de collaborer avec trois autres développeurs, ce qui a nécessité la mise en place d'outils efficaces pour le travail collaboratif. Nous avons opté pour l'utilisation de Notion, un outil en ligne polyvalent qui fonctionne comme un grand tableau blanc interactif. Notion nous a permis de centraliser toutes les informations relatives au projet, facilitant ainsi la communication et la collaboration entre les membres de l'équipe.

En adoptant une méthode Agile pour la gestion du projet, nous avons mis en place un tableau KANBAN sur Notion. Cet outil de gestion visuelle nous a permis de planifier, classer et hiérarchiser les tâches de manière transparente et organisée. Le tableau KANBAN nous a aidés à suivre l'avancement des tâches, à identifier les blocages éventuels et à maintenir un historique clair des activités. Cette approche nous a permis de nous adapter rapidement aux changements et de garantir que toutes les étapes du projet étaient bien coordonnées et suivies.

L'utilisation de Notion et du tableau KANBAN a grandement contribué à l'efficacité de notre collaboration et à la réussite du projet, en assurant une gestion fluide et structurée des tâches et en favorisant une communication continue et productive au sein de l'équipe.

EvoFlux - Serious Game

Board view Filter Sort + Search + New

KANBAN

- À faire** 0
 - + New
- En cours** 1
 - Dossier professionnel
 - Nadege Zongo
 - Clément Chevrier
 - Julien Fondu
 - T. Thomas
 - + New
- En test** 1
 - App Mobile React Native
 - Clément Chevrier
 - Julien Fondu
 - + New
- Termine** 21
 - Gestion des vidéos
 - Julien Fondu
 - Clément Chevrier
 - Création de l'avatar & images de scénarios
 - Thomas
 - Julien Fondu
 - Création et liaison de la base de données
 - Nadege Zongo
 - Clément Chevrier

Gestion du score par thématiques + Araignée (Radar Matplotlib)

- Status: Terminé
- Assign: Clément Chevrier
- Date_début: January 30, 2024
- Date_fin: February 10, 2024



V – GESTION DE PROJET

2. Réunions

Tout au long de la réalisation du projet, nous avons organisé des réunions régulières pour discuter de nos avancées, clarifier certains points et ajuster nos plans en fonction des besoins. Ces réunions ont été cruciales pour assurer une coordination efficace et pour résoudre rapidement tout problème ou ambiguïté.

Nous avons également collaboré étroitement avec EvoFlux, l'entreprise partenaire, en incluant leur point de vue et en les tenant informés de l'évolution du projet. Les réunions avec EvoFlux ont permis de recueillir des retours précieux et de garantir que le projet répondait aux attentes et aux exigences de l'entreprise.

Pendant la première semaine de rush, qui s'est déroulée au sein de Beauvoir, nous avons tenu des réunions courtes d'environ 15 minutes chaque jour. Cette fréquence élevée a facilité une communication rapide et efficace, permettant de gérer les tâches urgentes et de suivre les progrès en temps réel.

Après cette période intensive, nous avons ajusté la fréquence des réunions à environ 30 minutes, se tenant un mercredi sur deux. Cette fréquence permettait de maintenir une bonne communication tout en offrant suffisamment de temps entre les réunions pour progresser sur les tâches en cours et intégrer les retours reçus.

Ces réunions ont été un élément clé dans la gestion du projet, favorisant une collaboration continue, une réactivité rapide aux défis et une adaptation constante aux besoins du projet et aux attentes des parties prenantes.



V – GESTION DE PROJET

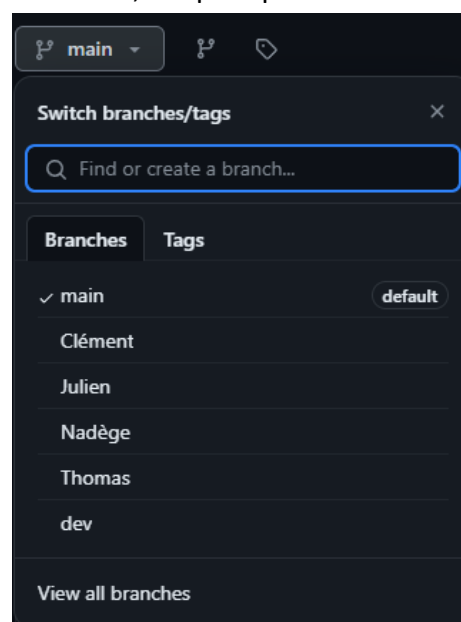
3. Versionning

Pour optimiser le développement du projet, nous avons adopté une approche de développement parallèle, permettant à chaque développeur de se concentrer sur des tâches spécifiques. Cette méthode a réduit le temps d'assemblage final et accéléré le processus de développement global. GitHub a été notre plateforme principale pour la gestion de version et la collaboration, facilitant le suivi des modifications et l'intégration fluide des contributions.

Chaque membre de l'équipe a travaillé sur des branches distinctes, ce qui a permis le développement simultané de fonctionnalités sans conflits majeurs. Les contributions ont été régulièrement consolidées dans la branche principale, maintenant ainsi une base de code cohérente et facilitant la résolution de conflits éventuels.

Nous avons également mis en place des hooks pré-commit via le système pre-commit. Ces hooks automatisent les vérifications de qualité du code, telles que l'analyse statique avec Flake8 et le formatage avec Black, garantissant ainsi que les contributions respectent les normes du projet avant d'être intégrées. Cela a amélioré la qualité du code et réduit les erreurs humaines, tout en simplifiant le processus d'intégration.

En combinant cette méthode de travail avec GitHub et les hooks pré-commit, nous avons optimisé la gestion du projet, amélioré la collaboration et assuré une intégration continue de haute qualité.





VI – RÉALISATION

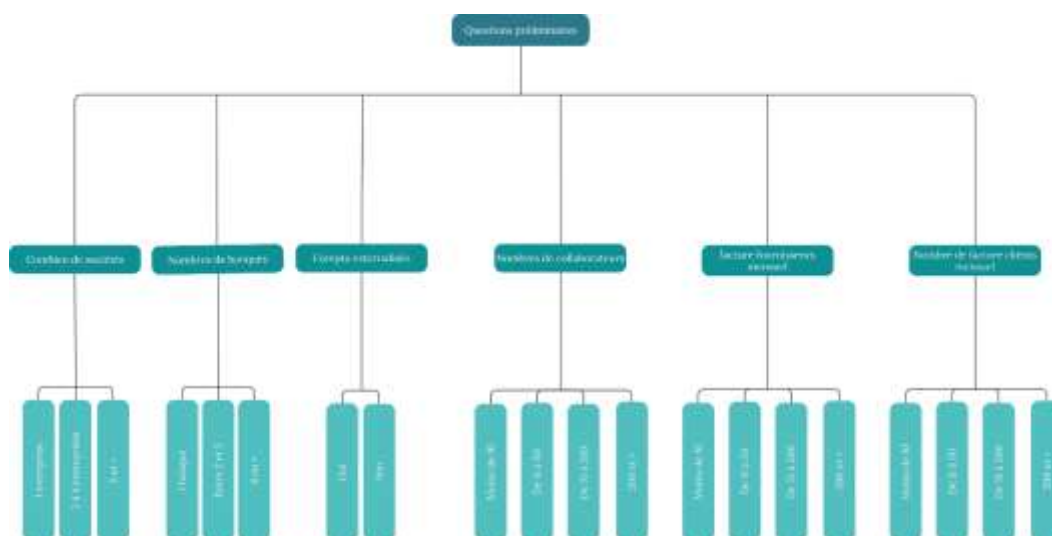
1. Scénarios & Visuels

Mise en place des scénarios :

Pour commencer la réalisation des différents visuels pour les scénarios de l'outil de diagnostic, nous avons organisé une réunion avec EvoFlux pour définir les questions et réponses nécessaires à l'évaluation de l'organisation de l'entreprise de l'utilisateur. Lors de cette réunion, nous avons répertorié et schématisé les questions pertinentes qui permettront de dresser un portrait détaillé des pratiques de gestion actuelles et d'identifier les sources potentielles d'amélioration.

Ces questions ont ensuite été classifiées en différentes thématiques afin de structurer l'évaluation de manière cohérente. Chaque réponse a été attribuée un score spécifique, ce qui nous permet de quantifier les différents aspects de la gestion de l'entreprise. Le score ainsi attribué sert à évaluer le potentiel de gain de temps pour le futur client, en fonction des différentes catégories de gestion, telles que la gestion commerciale, la comptabilité, la paie, etc.

Les schémas réalisés illustrent cette méthodologie en montrant comment les questions et réponses sont organisées et comment les scores sont calculés. Ils fournissent une vue d'ensemble claire de la manière dont l'outil de diagnostic évaluera et analysera les pratiques de gestion de l'utilisateur pour proposer des solutions adaptées et efficaces.





VI – RÉALISATION

Création des visuels :

Proposition 1 :

Dans un premier temps, nous avons décidé de confectionner l'avatar, les fonds ainsi que les audios et les animations grâce à divers outils.

❖ Avatar :

Nous avons réalisé divers avatars grâce à divers outils d'intelligence artificielle tel que Lexica et Adobe Firefly.



❖ Fonds :

Quant à eux, les fonds ont été créés grâce à des algorithmes de text-to-image disponibles gratuitement sur Hugging Face et notamment à celui de runway nommé stable-diffusion-v1-5 que nous avons jugé étant le plus fiable à travers plusieurs essais.





VI – RÉALISATION

❖ Audios :

Grâce à ElevenLabs, nous avons pu générer les différents dialogues des scénarios à partir d'un prompt contenant le script des paroles ainsi qu'à un extrait de voix dont nous avons décidé de prendre en tant que voix de notre avatar.

Après divers essais, grâce à des voix de personnalités plus ou moins connus, voir directement de celles des membres d'EvoFlux, le choix final de l'entreprise a été celui de prendre la voix d'Alexandre Astier.

 Q7.mp3 

 Q8.mp3 

 Q9.mp3 

❖ Animations :

Pour enrichir l'expérience utilisateur et rendre l'outil de diagnostic plus interactif, nous avons intégré des animations en utilisant la plateforme D-ID. Cette technologie permet d'animer des avatars et de synchroniser leur voix avec des fichiers audio.

Voici le rendu final des visuels :





VI – RÉALISATION

Proposition 2 :

Après la présentation des visuels réalisés à EvoFlux, il a été décidé d'abandonner ces visuels en raison de leur complexité de réalisation et du rendu jugé moyennement satisfaisant. Les visuels étaient trop compliqués à produire pour le résultat attendu.

En réponse à cette décision, nous avons proposé la solution Animaker, qui a immédiatement séduit EvoFlux. Animaker simplifie considérablement la création de visuels pour les scénarios, permettant une conception plus rapide et efficace. Grâce à cette solution, nous avons pu créer des visuels plus adaptés et fonctionnels pour l'outil de diagnostic.

Cependant, nous avons conservé les fichiers audio générés via ElevenLabs, qui restent une partie intégrante de l'expérience utilisateur. Ces fichiers audio peuvent être directement synchronisés avec les avatars dans Animaker, assurant ainsi une continuité et une cohérence dans la présentation des informations. Cette intégration permet de combiner la simplicité de création visuelle d'Animaker avec la qualité des audios préexistants, offrant ainsi une solution harmonieuse et efficace pour les besoins de l'outil de diagnostic.





VI – RÉALISATION

2. Accès aux données

Dans le cadre du projet, il a fallu intégrer des composants d'accès aux données via une connexion à la base de données SQL afin de pouvoir afficher les questions / réponses ainsi que d'enregistrer les données renseignées par l'utilisateur.

Pour les diverses applications réalisées, la base de données sera unique et sera stockée directement sur le serveur de l'entreprise situé dans une DMZ (sous-réseau isolé à la fois du réseau local et de l'internet).

Pour chacune des applications, les requêtes seront donc très similaires et utiliserons les opérations CRUD (CREATE, READ, UPDATE et DELETE), hormis l'opération « DELETE » qui nous sera d'aucunes utilités dans notre cas.

Voici des exemples de requêtes pour nos différentes applications :

❖ CREATE (INSERT) :

```
# Record the participant's choice, update Participation_Answers, and retrieve the ID of the next question
try:
    self.cursor.execute("INSERT INTO Participation_Answers (ID_Participation, ID_Theme, ID_Answer) VALUES (?, ?, ?)",
        (participation_id, theme_id, choice))
    self.conn.commit()

    self.cursor.execute("SELECT ID_Next_Question FROM Answers WHERE ID_Answer=?", (choice,))
    next_question_id = self.cursor.fetchone()[0]
```

❖ READ (SELECT) :

```
# Fetch question text, choices, and additional data from the Questions and Answers tables
self.cursor.execute("SELECT Content FROM Questions WHERE ID_Question=?", (question_number,))
question_text = self.cursor.fetchone()[0]
```

❖ UPDATE (UPDATE) :

```
# Update user information in the Users table
self.cursor.execute("""UPDATE Users SET Name = ?, Firstname = ?, Email = ?, Phone = ?,
    Company = ?, Function = ?, Conditions = ?, RGPD = ? WHERE ID_User = ?""",
    (ecr_name, ecr_firstname, ecr_email, ecr_phone, ecr_company, ecr_function, conditions, rgpd, user_id))
self.conn.commit()
```



VI – RÉALISATION

3. Application Desktop

Ressources utilisées :

L'application Desktop est entièrement réalisée en python et pour cela, nous avons utilisé différentes bibliothèques tel que :

- ❖ Tkinter, pour toute la base de l'interface graphique.
- ❖ Vlc, nous permettant de diffuser les vidéos des différents scénarios en arrière-plan.
- ❖ Pillow, qui nous permet quant à elle de mettre des images en arrière-plan de notre application.
- ❖ Matplotlib, pour l'affichage du score par thème sous forme de radar / araignée.
- ❖ Et Numpy, qui nous sert simplement à réaliser des calculs mathématiques lors de la création du radar.

```
import vlc
import numpy as np
from tkinter import *
from PIL import ImageTk, Image, ImageFilter
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
```

Pour ce qui en est des données, les tables contenant les questions, les thèmes ainsi que les réponses ont été renseignés dans des fichiers csv, ce qui nous a en premier lieu grandement facilité la tâche afin de re créer entièrement la base de données dès que nous en avons besoin.

Depuis, nous avons donc mise en place la base de données définitive sur le serveur d'EvoFlux, que nous avons créé à partir du script que nous avons précédemment réalisé pour la base de données en SQLite.

```
with open('data/questions.csv', 'r', newline='', encoding='utf-8') as file:
    csv_reader = csv.DictReader(file)
    for row in csv_reader:
        conn.execute("INSERT INTO Questions (ID_Question, Content, Video_Path, ID_Theme) VALUES (?, ?, ?, ?)",
                    (row['ID_Question'], row['Content'], row['Video_Path'], row['ID_Theme']))
```

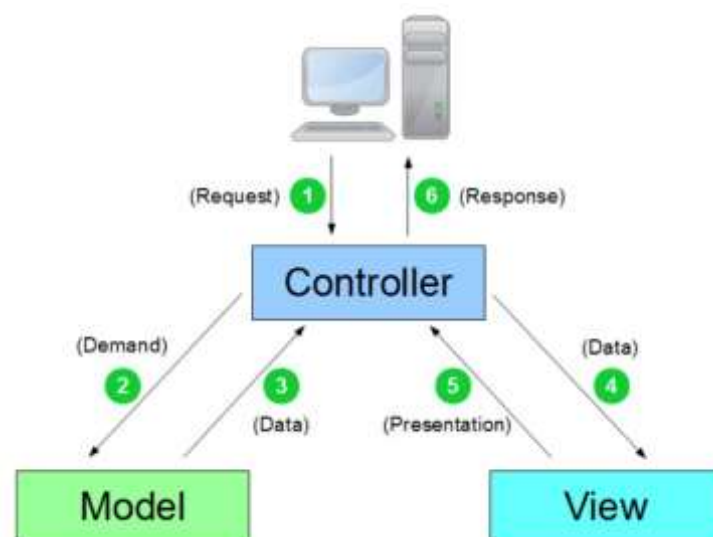
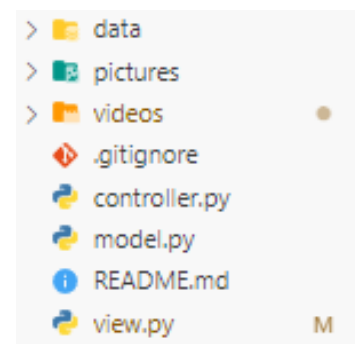


VI – RÉALISATION

MVC :

L'application est construite sur la structure d'un MVC (Model, View, Controller). Ce principe de développement consiste à diviser notre code en 3 parties :

- ❖ Model : Cette partie représente la structure logique des données. Ce modèle ne contient aucune information sur l'interface utilisateur et ne fait uniquement que requêtes / demandes à la base de données.
- ❖ View : Dans une seconde partie, nous retrouvons la vue qui est uniquement composée des éléments de l'interface utilisateur (tous ceux que l'utilisateur voit à l'écran et avec lesquels il peut interagir : boutons, boîtes de dialogue, etc.). Il regroupe le style, le texte et l'architecture complète de la partie visible de l'application.
- ❖ Controller : C'est la partie principale de l'application. Le controller est la partie du code qui permet de faire fonctionner l'ensemble de l'application, il contrôle l'ensemble des données qui provient des autres fichiers de l'application. Il sert à faire la communication entre les classes dans le modèle et la vue.





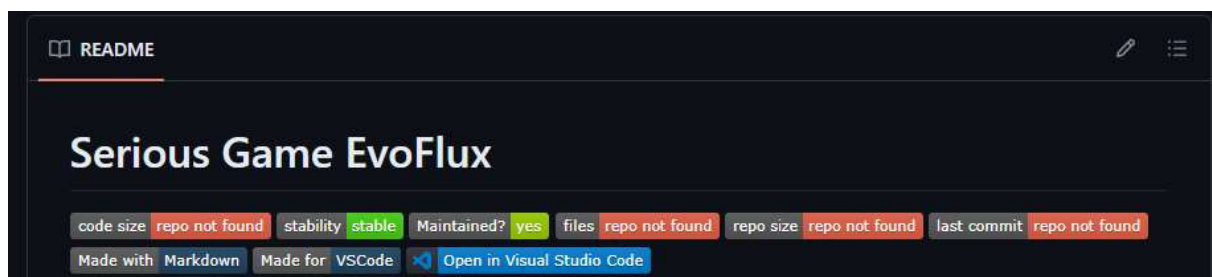
VI – RÉALISATION

Installation et documentation :

Script d'installation :

Pour que l'application soit le plus simple possible à utiliser / à installer, un fichier exécutable a été créé. Il suffit donc qu'à le lancer pour pouvoir l'utiliser.

Cependant, une documentation rapide d'installation du projet pour pouvoir l'exécuter depuis un outil de développement a été réalisée dans le fichier « README.md » et est disponible depuis le projet GitHub de l'application, qui est bien évidemment en privé et uniquement accessible à l'entreprise EvoFlux.



Code source commenté :

De plus, l'entièreté du code a été commenté, en anglais, pour qu'il puisse, dans un futur, être repris et modifié par un autre développeur ou voir même directement par les dirigeants d'EvoFlux.

En voici un exemple avec une partie du code constituant la vue :

```

145 # Function to show a page with questions in the Tkinter window
146 def show_page(self, image_number, participation_id):
147     global choice_buttons
148     global button_exit
149
150     # Stop the previous video
151     if View.vlc_player is not None:
152         View.vlc_player.stop()
153
154     # Fetch Datas
155     current_dir = os.path.dirname(os.path.realpath(__file__))
156     question_text, choices, video_path = Model().fetch_question_data(image_number)
157     choice_buttons, disabled_button = self.create_choice_buttons(choices, image_number-1, participation_id)

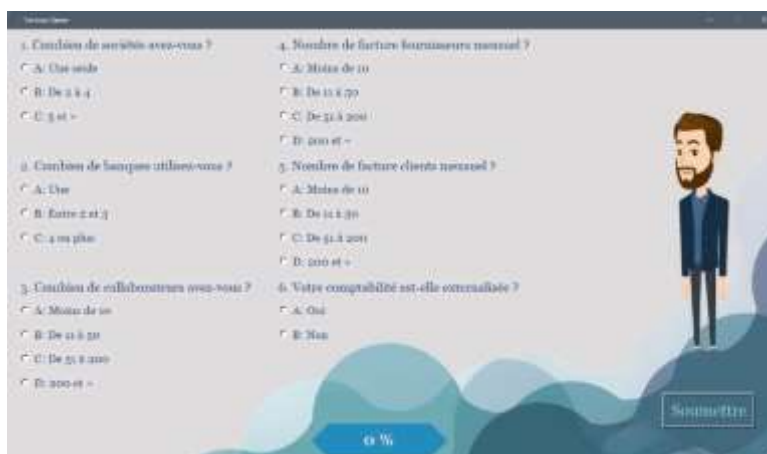
```




VI – RÉALISATION

Fonctionnement de l'application :

- ❖ En premier lieu, les utilisateurs arrivent sur la page de garde de l'outil.



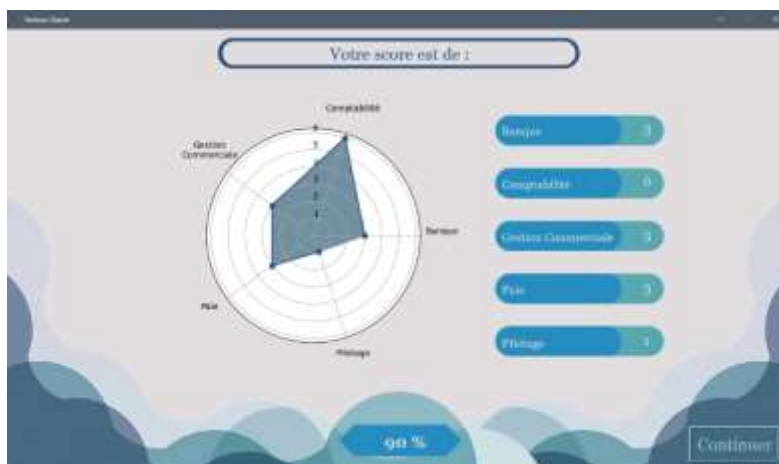
- ❖ Une fois après avoir cliqué sur le bouton « Commercer », 6 questions préliminaires sont posées afin de recenser les premières informations essentielles au diagnostic, avec l'avatar qui introduit rapidement le diagnostic.

- ❖ Ensuite, le reste des questions en page par page avec vidéo de fond animée par un avatar qui récite les questions, une barre de progression est présente pour que l'utilisateur sache à peu près où il en est.





VI – RÉALISATION



❖ Après avoir fini de répondre aux questions, la page suivante informe l'utilisateur du score de son entreprise en fonction des pistes d'améliorations qu'EvoFlux peut proposer par domaines, avec un aperçu du temps potentielle qui pourrait être gagnée de façon hebdomadaire / mensuel.

❖ Enfin, l'utilisateur est libre de pouvoir laisser ses informations personnelles s'il souhaite être recontacté par EvoFlux et approfondir les différentes pistes d'améliorations.



❖ Une fois ceci fait, il atterrit sur la page finale de remerciement.



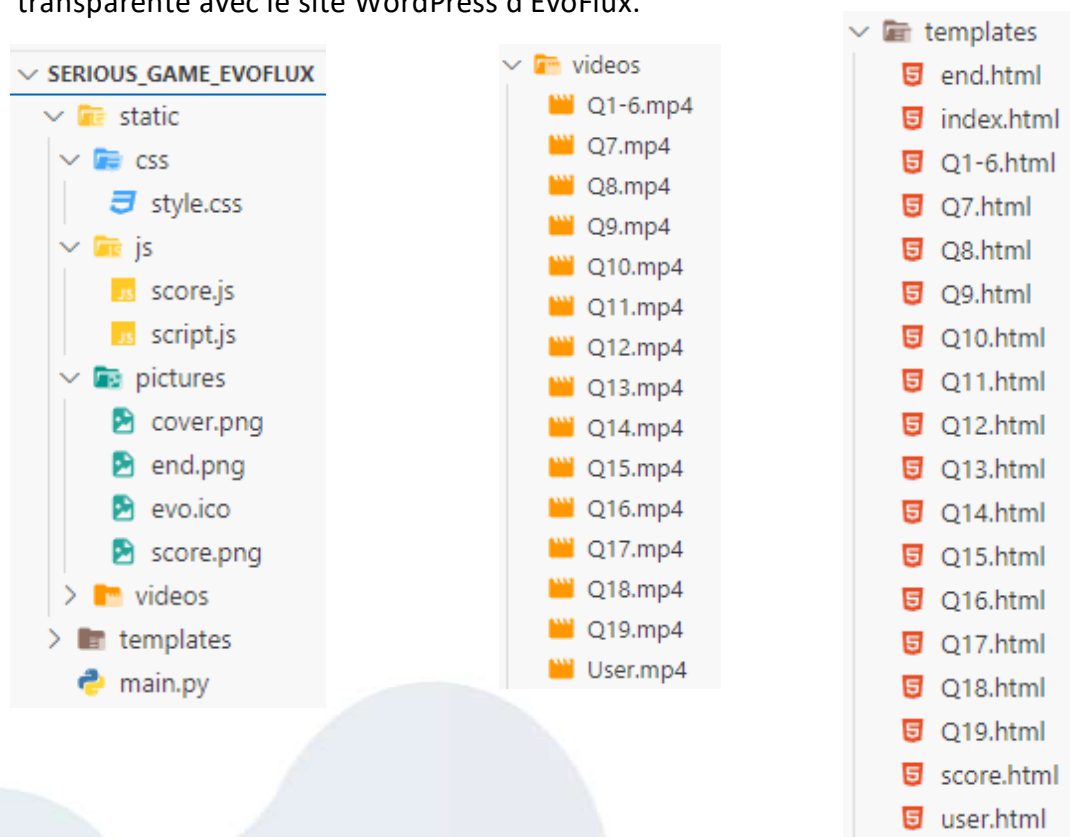
VI – RÉALISATION

4. Site Web

L'application Web constitue une reproduction fidèle de l'application Desktop. Elle intègre les mêmes fonctionnalités, visuels, et scénarios, et interagit avec la même base de données. Cette cohérence assure que les utilisateurs bénéficient d'une expérience utilisateur homogène, quel que soit le support utilisé.

Avant son intégration au site web WordPress d'EvoFlux, le projet est d'abord développé et testé en local. L'application Web est réalisée en utilisant Flask, un micro-framework pour Python, qui permet de construire des applications web de manière rapide et efficace. Grâce à Flask, le site web est structuré selon le modèle MVC (Model-View-Controller), un modèle de conception qui favorise une organisation claire et une séparation des préoccupations. Ce modèle rend le code bien structuré, facile à maintenir et réutiliser, et facilite les évolutions futures du projet.

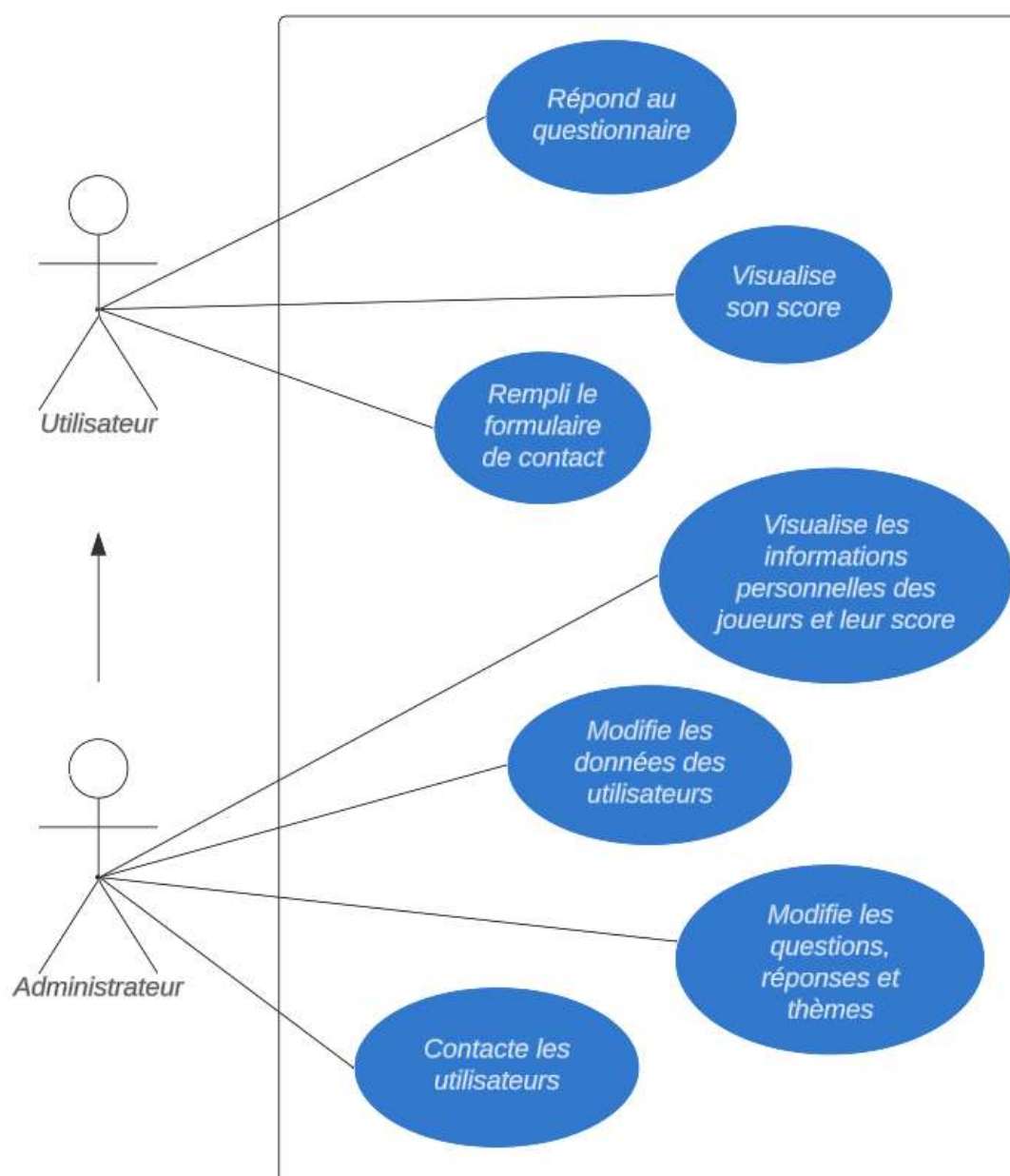
Cette approche garantit que l'application Web est non seulement fonctionnelle et cohérente avec l'application Desktop, mais aussi flexible et facile à adapter pour une intégration transparente avec le site WordPress d'EvoFlux.



VI – RÉALISATION

Diagramme de cas d'utilisation UML :

Le diagramme des cas d'utilisation ci-dessous présente les principales interactions entre les deux types d'utilisateurs de notre outil : les administrateurs et les clients. Il met en lumière les actions spécifiques que chaque groupe peut réaliser au sein de l'application, offrant ainsi une vue d'ensemble claire et concise des fonctionnalités disponibles pour chacun, tout en garantissant que l'outil reste adaptable, sécurisé et accessible pour tous.





VI – RÉALISATION

Front-End :

La partie Front-End du site web est conçue pour refléter l'apparence de l'application Desktop tout en offrant une expérience utilisateur améliorée. Bien que l'interface visuelle soit similaire à celle de l'application Desktop, le fonctionnement technique est différent. Pour chaque question, une page HTML distincte (template) est utilisée.

Chaque page HTML est soigneusement stylisée en utilisant des fichiers CSS et enrichie avec des scripts JavaScript. Ces éléments permettent de créer des interfaces visuellement attrayantes et dynamiques, améliorant ainsi l'expérience utilisateur. Les styles CSS assurent une présentation cohérente et professionnelle, tandis que les scripts JavaScript ajoutent des interactions et des animations pour rendre l'application plus engageante et réactive.

```

templates > score.html > ...
You, hier | 1 author (You)
1  <!DOCTYPE html>
2  <html lang="fr">
3    <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Serious Game EvoFlux</title>
7      <link rel="stylesheet" href="/static/css/style.css">
8      <link rel="icon" type="image/icon" href="/static/pictures/evo.ico">
9      <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
10   </head>
11   <body>
12     <div class="image-container">
13       
14     </div>
15
16     <div class="radar-chart">
17       <canvas id="myChart"></canvas>
18       <div id="scores-container"></div>
19     </div>
20
21     <div class="question-label" style="left: 48%; font-size: 25px">Score :</div>
22
23     <button class="leave-button" style="left: 75%" onclick="window.location.href='/user'">Continuer</button>
24
25     <div class="progress-label" style="top: 82%">90 %</div>
26
27     <script src="{{ url_for('static', filename='js/score.js') }}"></script>
28
29   </body>
30 </html>

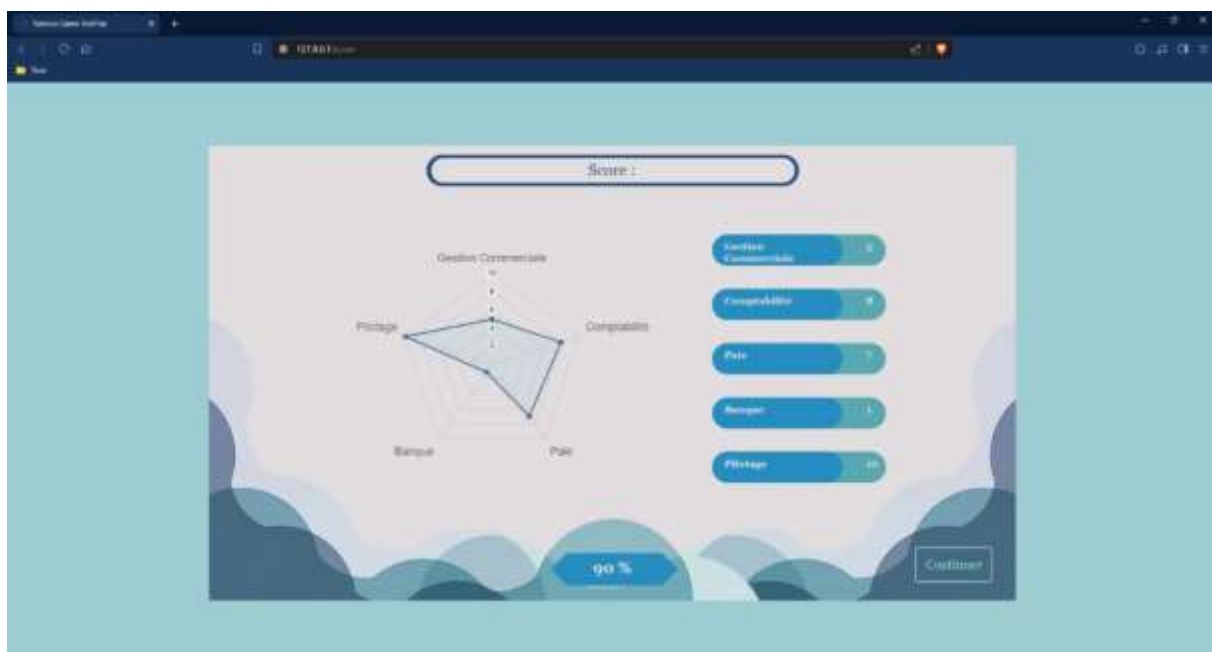
```



VI – RÉALISATION

Voici à quoi ressemble la page de score, qui démontre comment le code HTML, les fichiers CSS et les scripts JavaScript se combinent pour offrir une présentation harmonieuse et fonctionnelle. Pour les pages de questions, nous avons maintenu la même vidéo de fond que celle utilisée dans l'application Desktop, assurant ainsi une continuité visuelle entre les deux versions de l'outil. Cette approche garantit que l'expérience utilisateur reste cohérente et immersive, que l'on utilise l'application Desktop ou le site web.

```
static > css > # style.css > start-button: hover
1  /* style.css */
2
3  body {
4    margin: 0;
5    padding: 0;
6    background-color: #9dcdd1;
7    font-family: Georgia, serif;
8    display: flex;
9    justify-content: center;
10   align-items: center;
11   height: 100vh;
12   position: relative;
13 }
14
15 .image-container {
16   position: relative;
17   width: 1280px;
18   height: 720px;
19 }
48  for (var i = 0; i < data.length; i++) {
49    var score = data[i];
50    var label = myChart.data.labels[i];
51
52    // Créer la balise h3 pour le nom du thème
53    var themeH3 = document.createElement('h3');
54    themeH3.textContent = label;
55    themeH3.style.fontFamily = 'Georgia';
56    themeH3.style.fontSize = '16px';
57    themeH3.style.color = '#E2DEDF';
58    themeH3.style.position = 'absolute';
59    themeH3.style.left = '120%';
60    themeH3.style.top = yPosition + (i*17) + '%';
61    scoresContainer.appendChild(themeH3);
```





VI – RÉALISATION

Back-End :

Le Back-End du site web correspond à la partie "Controller" du modèle MVC (Model-View-Controller). Il constitue le cœur du fonctionnement du site, en orchestrant la gestion des requêtes et des interactions entre le modèle, la vue, et les autres composants de l'application. Il joue un rôle crucial en importation des bibliothèques nécessaires au fonctionnement du site. Il définit et gère toutes les routes (URLs) disponibles, qui permettent de naviguer vers les différentes pages du site. Chaque route est associée à une fonction spécifique du contrôleur, qui traite les requêtes entrantes, récupère les données nécessaires, et transmet les informations appropriées à la vue pour affichage.

Ce fichier importe donc les bibliothèques nécessaires et répertorie toutes les routes (URLs) possibles qui nous emmènent sur les diverses pages du site.

```
main.py > Q12
You, hier | 1 author (You)
1 from flask import Flask, render_template
2
3 app = Flask(__name__)
4
5 @app.route('/')
6 def index():
7     return render_template('index.html')
8
9 @app.route('/Q1-6')
10 def Q1_6():
11     return render_template('Q1-6.html')
12
13 @app.route('/Q7')
14 def Q7():
15     return render_template('Q7.html')
16
17 @app.route('/score')
18 def score():
19     return render_template('score.html')
20
21 @app.route('/user')
22 def user():
23     return render_template('user.html')
24
25 @app.route('/end')
26 def end():
27     return render_template('end.html')
28
29 if __name__ == '__main__':
30     #app.run(debug=True)
31     app.run(port=80, debug=True)
```

Le « Controller » contient bien évidemment les paramètres de démarrage et d'exécution du code. Comme nous pouvons le voir ci-dessous, le site se donc disponible sur l'adresse IP par défaut, qui est l'adresse IP locale 127.0.0.1 et sur le port 80, puisque nous en avons décidé ainsi (de base, un projet Flask s'exécute sur le port 5000).

```
* Serving Flask app 'main'
* Debug mode: on
WARNING: This is a development server.
instead.
* Running on http://127.0.0.1:80
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 432-743-452
```




VI – RÉALISATION

5. Application Mobile

Pour répondre à la demande du client, qui souhaitait mettre à disposition un petit formulaire de contact simple, nous avons choisi de développer l'application mobile en React Native avec Expo Go. Cette décision a été prise pour sa simplicité d'accessibilité et sa facilité de développement cross-platform.

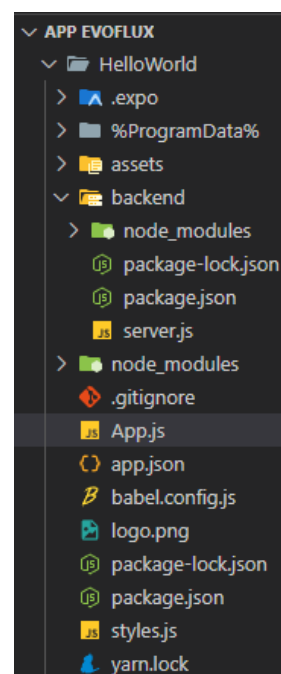
L'application est construite sous le format suivant :

- ❖ **Serveur Node.js** : Le serveur Node.js gère toute l'interaction avec la base de données. Il est responsable de la gestion des données, des requêtes de l'application mobile, et des opérations nécessaires pour stocker et récupérer les informations des utilisateurs.

```
PS C:\Users\Clément\Desktop\App EvoFlux\HelloWorld\backend> node server.js
Server running at http://localhost:3000
```

- ❖ **Fichier style.js** : Ce fichier est dédié à la gestion du visuel de l'application. Il contient les styles CSS nécessaires pour la présentation et l'interface utilisateur, assurant ainsi une apparence cohérente et attrayante de l'application mobile.
- ❖ **Fichier App.js** : Ce fichier est le point d'entrée principal de l'application. Il initialise et configure le fonctionnement global de l'application mobile, assurant le bon démarrage et la gestion des différentes fonctionnalités et écrans.

```
PS C:\Users\Clément\Desktop\App EvoFlux\HelloWorld> npx expo start
Starting project at C:\Users\Clément\Desktop\App EvoFlux\HelloWorld
```





VI – RÉALISATION

Une fois mise en service, l'application mobile est disponible sur les plateformes Android et iOS, offrant ainsi une large accessibilité aux utilisateurs sur différents dispositifs.

Le formulaire de contact de l'application, conçu pour être simple et fonctionnel, respecte la charte graphique souhaitée par le client. Ce formulaire est conçu pour être intuitif et facile à utiliser, avec une interface qui reflète l'identité visuelle de l'entreprise.

Voici à quoi ressemble le formulaire de contact, qui reprend la charte graphique souhaitée :

```

41 // Endpoint to fetch all submissions
42 app.get('/submissions', (req, res) => {
43   db.all("SELECT * FROM submissions", [], (err, rows) => {
44     if (err) {
45       return res.status(500).json({ error: err.message });
46     }
47     res.status(200).json({ submissions: rows });
48   });
49 });

```

Les données enregistrées par le formulaire de contact sont visibles ici et sont stockées dans la même base de données que celle utilisée par le site web et l'application desktop. Cette intégration assure une gestion centralisée et cohérente des informations à travers toutes les plateformes.

```

{
  "submissions": [
    {
      "name": "chevrier",
      "firstName": "clément",
      "email": "clement.chevrier@gmail.com",
      "phone": "06 11 11 11 11",
      "company": "EvoFlux",
      "position": "Data Engineer",
      "message": "Test"
    }
  ]
}

```



VI – RÉALISATION

6. Tests et Sécurité :

Tests unitaires :

Afin de vérifier notre code nous avons réalisé divers tests unitaires sur certaines fonctions de nos applications.

En plus de valider le bon fonctionnement des fonctions et de simplifier le code, les tests unitaires jouent un rôle crucial dans le processus de développement logiciel. Ils permettent d'identifier et de corriger les erreurs dès leur apparition, ce qui contribue à réduire les coûts et les risques associés aux bugs logiciels.

De plus, en écrivant des tests unitaires, les développeurs documentent implicitement le comportement attendu des fonctions, ce qui facilite la compréhension et la maintenance du code à long terme.

En intégrant les tests unitaires dans les processus de développement, les équipes peuvent également gagner en confiance dans la qualité du logiciel produit, favorisant ainsi une livraison plus rapide et plus fiable des fonctionnalités aux utilisateurs finaux.

Ici, nous vérifions grâce à Unittest (une bibliothèque python), que le processus de création d'une nouvelle participation au questionnaire se déroule comme souhaitée, sans erreurs et surtout en un minimum de temps possible.

```
# Test: Create a new participation
def test_generate_unique_participation_id(self):
    user_id = self.model.generate_unique_user_id('Doe', 'Jane', 'jane@example.com', '987654321', 'CompanyY', 'Manager')
    participation_id = self.model.generate_unique_participation_id(user_id)
    self.assertIsNotNone(participation_id)
    self.model.cursor.execute("SELECT * FROM Participation WHERE ID_Participation = ?", (participation_id,))
    participation = self.model.cursor.fetchone()
    self.assertIsNotNone(participation)
    self.assertEqual(participation[1], user_id)
```

```
PS C:\Projets\Serious_Game_EvoFlux> python -m unittest test_model.py
```

```
..
```

```
-----
Ran 2 tests in 0.003s
```

```
OK
```



VI – RÉALISATION

Sécurité :

RGPD et Droits de réutilisation de données :

Pour garantir la conformité avec le Règlement Général sur la Protection des Données (RGPD), qui encadre le traitement des données personnelles dans l'Union Européenne depuis 2018, des mesures spécifiques ont été mises en place. Le RGPD vise à assurer un niveau élevé de protection des données personnelles en harmonisant les réglementations au sein de l'UE et en renforçant les droits des individus sur leurs données.

Dans cette optique, l'application intègre une fonctionnalité essentielle : une case à cocher (« checkbox ») sur la page des informations utilisateurs. Cette option permet à l'utilisateur de choisir s'il souhaite ou non fournir et enregistrer ses données personnelles. Si l'utilisateur choisit de ne pas fournir ses informations personnelles, les réponses aux questions seront alors enregistrées de manière anonyme.

Cette approche garantit que les données personnelles ne sont collectées que lorsque l'utilisateur donne son consentement explicite, en accord avec les exigences du RGPD. En cas de refus, la protection de la vie privée de l'utilisateur est assurée par l'anonymisation des réponses, respectant ainsi les droits des individus tout en permettant le traitement des données dans le cadre des objectifs de l'application.

Cette fonctionnalité est essentielle pour garantir la conformité réglementaire et renforcer la confiance des utilisateurs dans la gestion de leurs données personnelles.

☐ J'accepte l'utilisation de mes informations
uniquement dans le but d'analyses ou afin d'être
recontacté.

```
if rgpd:
    # Update User Info into the Database if rgpd is check
    Model().update_user_info(user_id, ecr_name, ecr_firstname, ecr_email,
    | | | | | | | | ecr_phone, ecr_function, ecr_company, rgpd)
    print("Les informations de l'utilisateur ont été mises à jour avec succès.")
```



VI – RÉALISATION

Injections SQL :

Pour prévenir les tentatives d'injection SQL, nous avons mis en place plusieurs mesures de sécurité afin de protéger l'intégrité et la confidentialité de notre base de données.

Premièrement, nous avons opté pour la saisie des URLs en dur, ce qui empêche les utilisateurs de modifier les paramètres de la requête via l'URL. Cette pratique assure que seules les URLs prédéfinies et sécurisées sont accessibles, réduisant ainsi le risque d'accès non autorisé aux pages et aux opérations de la base de données.

De plus, nous avons implémenté des restrictions strictes sur les entrées des utilisateurs et les erreurs potentielles de saisie. Pour ce faire, nous avons utilisé des expressions régulières (regex) et des vérifications de la longueur des champs de texte libre (« Entry »). Ces mécanismes de validation assurent que les données saisies respectent les formats attendus et réduisent les risques d'injection malveillante.

Par exemple, la regex utilisée pour les adresses e-mail, nommée « email_regex », est conçue pour vérifier que le champ renseigné respecte la structure d'une adresse e-mail valide. Cette regex s'assure que l'adresse contient uniquement des lettres majuscules et minuscules ainsi que des chiffres avant le symbole « @ », et que la partie après le « @ » se termine par un domaine valide avec au moins deux caractères après le point final (comme .com, .fr, etc.). Cette vérification rigoureuse limite les entrées au strict nécessaire et évite les tentatives d'injection via des données malformées.

Ces pratiques de sécurité sont essentielles pour maintenir l'intégrité de la base de données et garantir une expérience utilisateur sécurisée et fiable. Elles contribuent à protéger le système contre les attaques potentielles tout en assurant que les données reçues sont conformes aux attentes.

```
# Check the length entry
length = len(text)
if int(min_length) <= length <= int(max_length):
    if is_email:
        # Check if the email is in a valid format using a regex
        email_regex = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
        if re.match(email_regex, text):
            return True
```

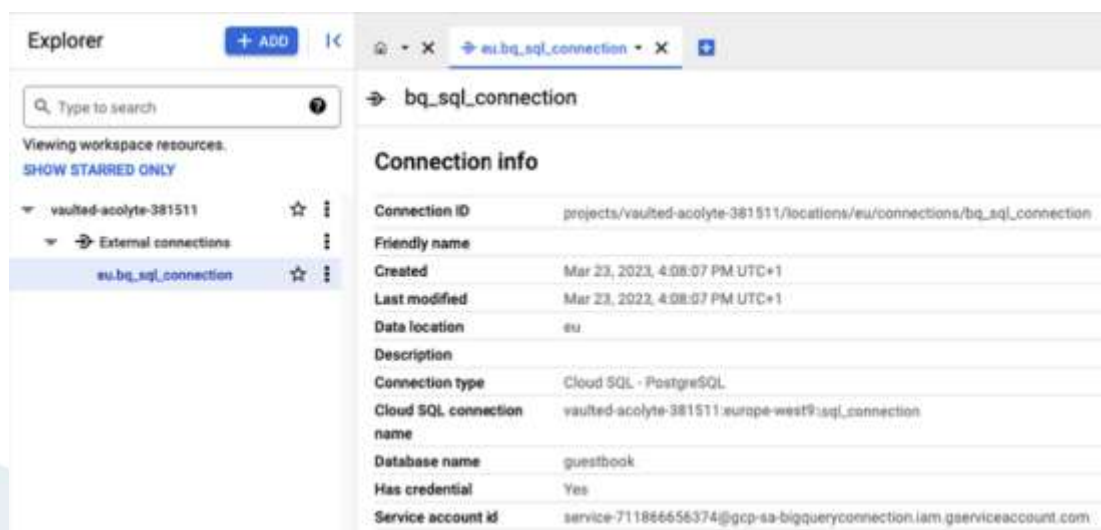


VI – RÉALISATION

7. Déploiement

Après une consultation approfondie avec EvoFlux et en réponse aux exigences de la RNCP, nous avons intégré le déploiement de la base de données sur Google Cloud Platform (GCP). Cette solution permet d'éviter le stockage local des données en les transférant directement vers un environnement cloud sécurisé. En optant pour GCP, nous bénéficions des avantages significatifs d'une infrastructure cloud moderne, notamment une scalabilité flexible, une haute disponibilité et des fonctionnalités avancées de sécurité et de sauvegarde.

La gestion de la base de données est assurée par le système de gestion de base de données PostgreSQL, déployé sur les serveurs de Google. PostgreSQL, reconnu pour sa robustesse et ses capacités de gestion de données complexes, offre des performances optimisées et une intégrité accrue des données. Grâce à cette solution, nous avons pu garantir une gestion efficace et sécurisée des données tout en minimisant les besoins en maintenance et en infrastructure locale. Cette approche cloud-first assure également une meilleure résilience et une réduction des coûts associés à la gestion des infrastructures sur site.





VI – RÉALISATION

Le déploiement de notre projet a été conçu pour garantir une accessibilité maximale à nos utilisateurs, en offrant des solutions adaptées à différents types de plateformes, tout en assurant une expérience utilisateur fluide et cohérente.

C'est pour cela que nous avons anticiper le déploiement de chaque solution :

❖ Application Desktop

L'application Desktop a été soigneusement compilée en un exécutable pour une installation facile et rapide sur les ordinateurs des utilisateurs. Cet exécutable sera mis à disposition via un lien de téléchargement direct, accessible sur le site web d'EvoFlux ainsi que sur leurs réseaux sociaux. Ce choix permet non seulement de simplifier le processus d'installation pour les utilisateurs, mais aussi de centraliser la distribution du logiciel, facilitant ainsi les mises à jour et la gestion des versions. Les utilisateurs pourront télécharger le fichier puis l'exécuter, sans qu'il soit nécessaire d'avoir des compétences techniques avancées. Cette approche assure que l'application est facilement accessible à un large public tout en étant sécurisée et fiable.

❖ Interface Web

Pour offrir une solution accessible directement depuis un navigateur, l'interface web de l'application a été intégrée au site WordPress existant d'EvoFlux. Cette intégration garantit une expérience utilisateur homogène, où les utilisateurs peuvent accéder aux fonctionnalités de l'application sans quitter le site qu'ils connaissent déjà. De plus, cette solution permet une maintenance centralisée, où les mises à jour de l'interface web peuvent être déployées sans nécessiter d'intervention de la part des utilisateurs finaux.

❖ Application Mobile

Conscients de l'importance d'une présence mobile, nous avons développé une version de l'application compatible avec les deux principales plateformes mobiles : iOS et Android. L'application sera publiée sur l'App Store pour les utilisateurs d'iPhone et sur le Google Play Store pour les utilisateurs de smartphones Android. Ce double déploiement garantit que l'application est accessible à la majorité des utilisateurs de smartphones. Le processus de publication sur ces plateformes implique des vérifications rigoureuses pour s'assurer que l'application répond aux standards de qualité et de sécurité imposés par Apple et Google. Une fois disponible sur ces stores, les utilisateurs pourront facilement rechercher, télécharger et installer l'application directement depuis leurs appareils.